UCC UNIVERSIDAD CATÓLICA DE CÓRDOBA JESUITAS

Calidad de Software

Mag. Pablo Virgolini

7 de septiembre de 2026

Proceso de desarrollo de pruebas

La obtención de casos de prueba debe partir directamente de los requisitos del sistema. Según el estándar **IEEE 829 (Test Documentation)**, cada caso de prueba debe documentar elementos específicos para garantizar su efectividad.

01

Inputs y Precondiciones

Datos de entrada y estado inicial requerido para ejecutar el caso de prueba.

02

Resultados Esperados

Comportamiento y salidas que debe producir el sistema ante esos datos.

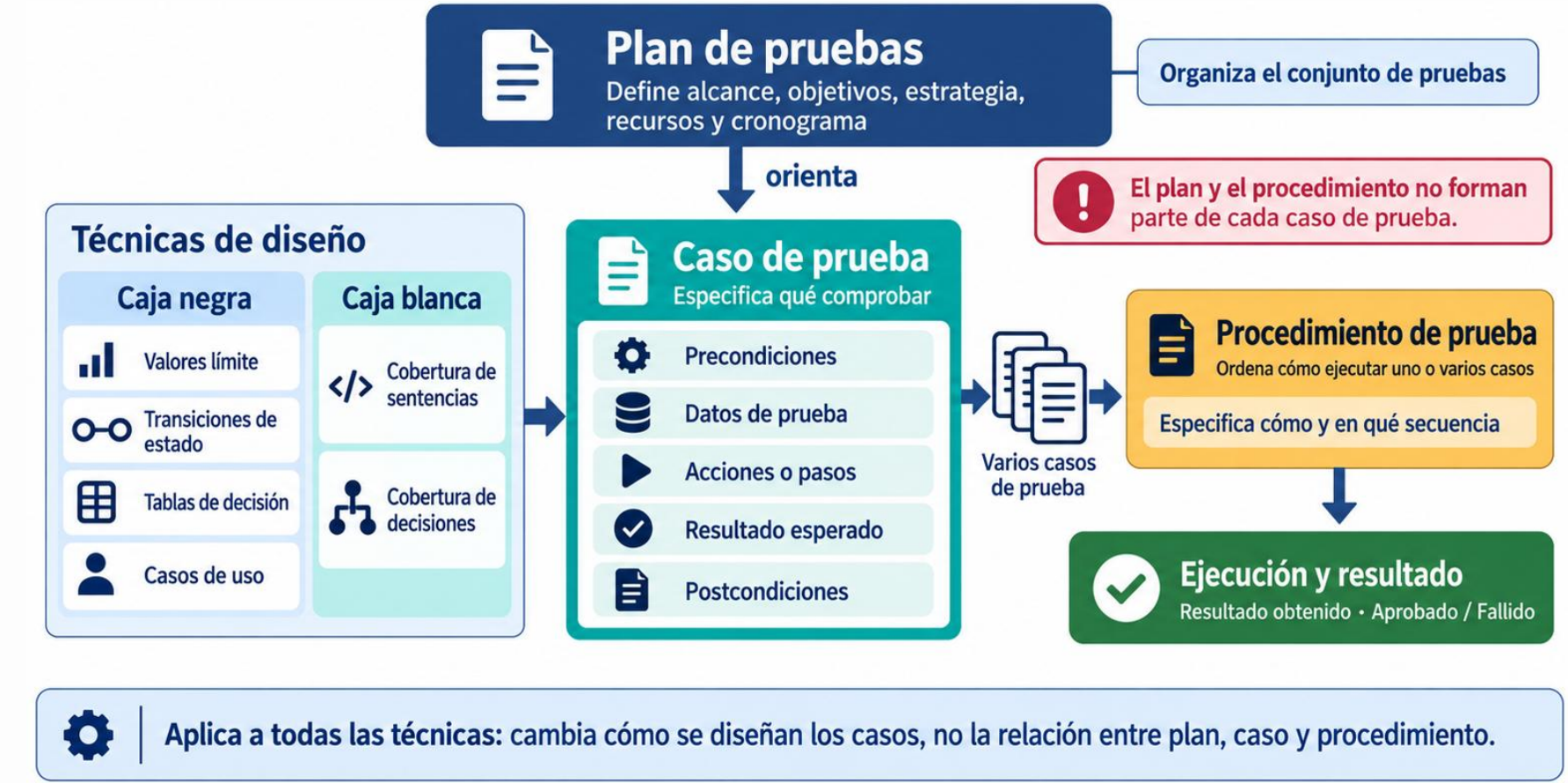
03

Postcondiciones y Dependencias

Estado final del sistema y relación con otros casos de prueba.

Trazabilidad: cada caso de prueba debe estar vinculado a un requisito específico para poder medir la cobertura de pruebas frente a la especificación.

Proceso de desarrollo de pruebas



Caso de prueba documentado

Requisito: el sistema debe entregar el pedido solo cuando el código presentado sea válido.

Campo	Contenido del caso de prueba
ID	**CP-RET-03**
Requisito vinculado	RF-03 — Validación del código de retiro
Escenario	**Flujo de excepción: código incorrecto**
Precondición	Pedido existente y en estado «Listo para retirar»
Datos de prueba	Código esperado: CAF-482 | Código ingresado: **CAF-428**
Pasos	1. Buscar el pedido 2. Ingresar CAF-428 3. Confirmar el retiro
Resultado esperado	El sistema informa «Código incorrecto» y no registra la entrega
Postcondición	El pedido permanece en estado **«Listo para retirar»**

El caso documenta qué se prueba, con qué datos, qué se ejecuta y qué resultado debe observarse.

Categorías de técnicas

Caja Negra

Se diseña sin mirar el código interno del sistema.

Enfoque: Prueba funcional basada en especificaciones.

Caja Blanca

Se diseña analizando la estructura interna del código.

Enfoque: Prueba estructural y cobertura de código.

Basadas en Experiencia

Utilizan el conocimiento y la experiencia del tester.

Enfoque: Pruebas exploratorias y predicción de errores.

Vamos a profundizar en **caja negra** y **caja blanca:** las bases de cualquier estrategia de pruebas sólida.

Técnicas de caja negra

Técnicas basadas en especificación: se enfocan en el comportamiento externo del sistema, sin considerar su implementación interna.

CE

Clases de Equivalencia

Agrupar entradas válidas e inválidas en categorías que se comportan de manera similar.

VL

Análisis de Valores Límite

Probar en los extremos de los rangos, donde suelen ocurrir errores.

TD

Tablas de Decisión

Combinaciones de condiciones para validar reglas de negocio complejas.

TE

Transición de Estados

Modelar cambios de estado del sistema y los eventos que los provocan.

CU

Casos de Uso

Escenarios completos de usuario, desde el inicio hasta el final.

Clases de equivalencia

- Es la porción del dominio de una entrada o una salida para la cual se asume que el comportamiento del sistema, según la especificación, es el mismo.

- Las pruebas se ejecutan utilizando un único representante de cada clase de equivalencia (CE).

- Las clases se escogen tanto para entradas (inputs) válidas como no válidas.

CE válida

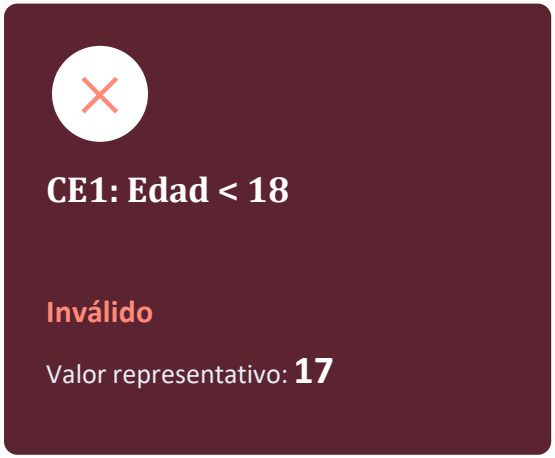
Todos los valores del rango de definición se combinan en una única clase, si son tratados de forma idéntica por el objeto de prueba.

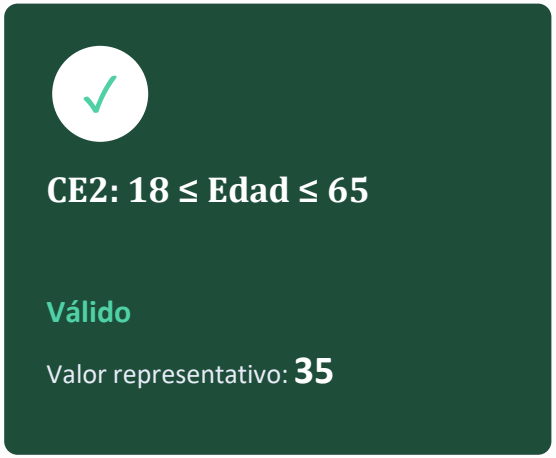
CE no válida

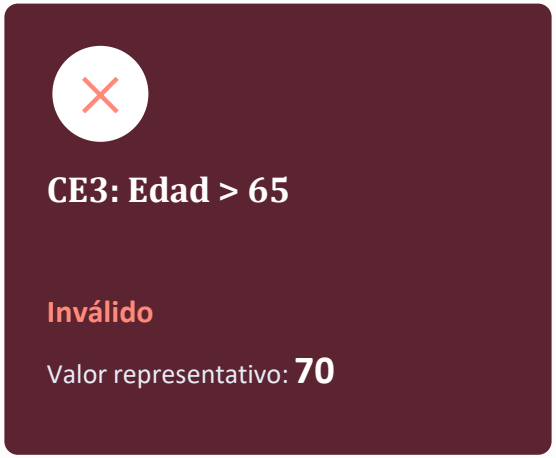
Se distinguen dos casos fuera del rango: valores con formato correcto pero fuera de rango, y valores con formato erróneo en una CE independiente.

Clases de equivalencia

Requisito: rango de edad válido entre 18 y 65 años (inclusive), para inscribirse a una beca.







Con 3 valores (17, 35, 70) cubrimos las 3 clases posibles: no hace falta probar cada edad entre 18 y 65.

Análsis de valores límite

- Los valores límite son especialmente importantes porque es donde suelen ocurrir los errores más comunes.

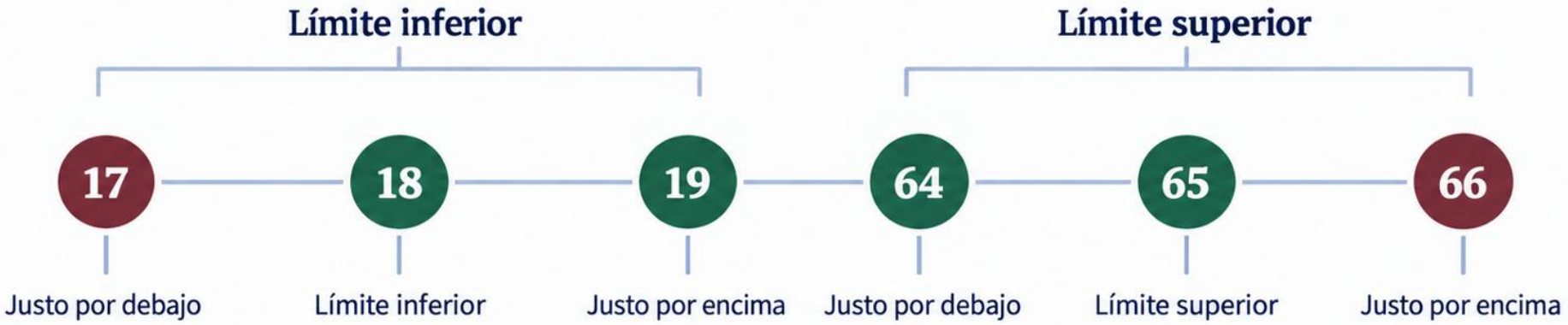
- Frecuentemente los límites del rango de valores no están bien definidos o conducen a distintas interpretaciones.

- Puede aplicarse en todos los niveles de prueba: es fácil de aplicar y su capacidad de detección de defectos es alta.

Ejemplo: un validador que usa "edad < 18" en vez de "edad ≤ 17" rechaza por error a una persona de exactamente 18 años en el límite. Por eso siempre probamos el valor límite y sus dos vecinos inmediatos (uno adentro, uno afuera del rango).

Análsis de valores límite

Requisito: rango de edad válido entre 18 y 65 años (inclusive).



En total: 6 casos de prueba — 4 dentro del rango válido (18, 19, 65 y 65) y 2 fuera de rango (17 y 66).

Tablas de decisión

Es una técnica de diseño en la que los casos de prueba se diseñan para ejecutar las combinaciones de entradas y/o estímulos (causas) representadas en una tabla de decisión.

1 Condiciones

Las entradas o causas que pueden variar (ej: saldo, receptor, token).

2 Combinaciones

Cada fila cruza los valores posibles de todas las condiciones.

3 Acciones

El resultado o efecto esperado del sistema para esa combinación.

- Ayuda a encontrar diferencias con los requisitos, en cualquier nivel de pruebas.
- Con el gráfico causa-efecto la cantidad de combinaciones posibles puede reducirse a un subconjunto sistemático.

Tablas de decisión

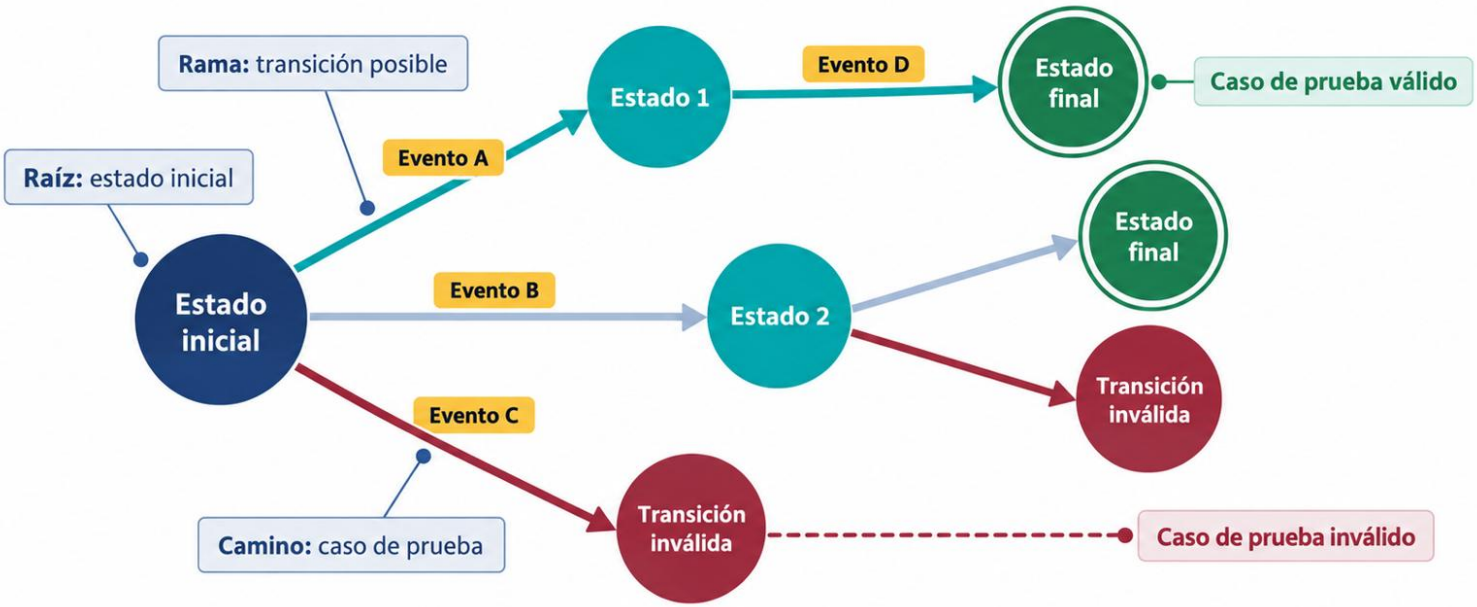
Caso: proceso bancario que autoriza transferencias, según saldo, receptor y token de seguridad.

Caso	Saldo suficiente	Receptor válido	Token válido	Resultado
1	Sí	Sí	Sí	**Autorizar**
2	No	Sí	Sí	**Rechazar**
3	Sí	No	Sí	**Rechazar**
4	Sí	Sí	No	**Rechazar**
…				

Con 3 condiciones booleanas hay 8 combinaciones posibles (2^3). La tabla de decisión asegura que probamos las combinaciones relevantes del negocio, no solo el “camino feliz” (caso 1).

Transición de estados

Los casos de prueba se diseñan para ejecutar transiciones de estado válidas e inválidas: una transición se inicia mediante la ejecución de un evento, y describe los cambios de estado de un objeto o componente.



Termina cuando: el nodo es un estado final (hoja del árbol), o el mismo nodo con el mismo estado ya es parte del árbol.

Transición de estados



Caso: árbol de transición de un pedido en una cafetería

Estado 1	Estado 2	Estado 3	Estado 4	Estado 5	Estado final
Pedido recibido	Aceptado	En preparación	Listo para retirar	Entregado	**Entregado**
Pedido recibido	Aceptado	En preparación	Listo para retirar	Cancelado	**Cancelado**
Pedido recibido	Aceptado	En preparación	Cancelado	—	**Cancelado**
Pedido recibido	Aceptado	Cancelado	—	—	**Cancelado**

Cada fila representa un camino distinto <<Pedido recibido>> hasta una hoja del árbol: 4 caminos, 4 casos de prueba.

Transición de estados



Caso: árbol de transición de un pedido en una cafetería

Estado 1	Estado 2	Estado 3	Estado 4	Estado 5	Estado final
Pedido recibido	Aceptado	En preparación	Listo para retirar	Entregado	**Entregado**
Pedido recibido	Aceptado	En preparación	Listo para retirar	Cancelado	**Cancelado**
Pedido recibido	Aceptado	En preparación	Cancelado	—	**Cancelado**
Pedido recibido	Aceptado	Cancelado	—	—	**Cancelado**

Cada fila representa un camino distinto <<Pedido recibido>> hasta una hoja del árbol: 4 caminos, 4 casos de prueba.

Pruebas de casos de uso

Los casos de prueba se obtienen de los escenarios que describen la interacción entre los actores y el sistema.



Un caso de uso describe la interacción de todos los actores involucrados, conduciendo a un resultado final por parte del sistema. Todo caso de uso tiene precondiciones que deben cumplirse para poder ejecutarlo, y postcondiciones que describen el estado del sistema tras su ejecución.

Caso: Knight Capital (2012)

- Knight Capital iba a incorporar una nueva funcionalidad en su sistema de enrutamiento de órdenes, y debía actualizar el software en 8 servidores.
- El despliegue se realizó correctamente en 7 servidores, pero el octavo conservó una versión anterior del código.
- Ese servidor reutilizó una bandera de configuración para la nueva funcionalidad, y por error activó "Power Peg": una rutina de trading obsoleta que llevaba años sin usarse ni probarse.
- Durante 45 minutos, el sistema generó órdenes de compra y venta automáticas e incorrectas sobre 150 acciones, a gran velocidad.

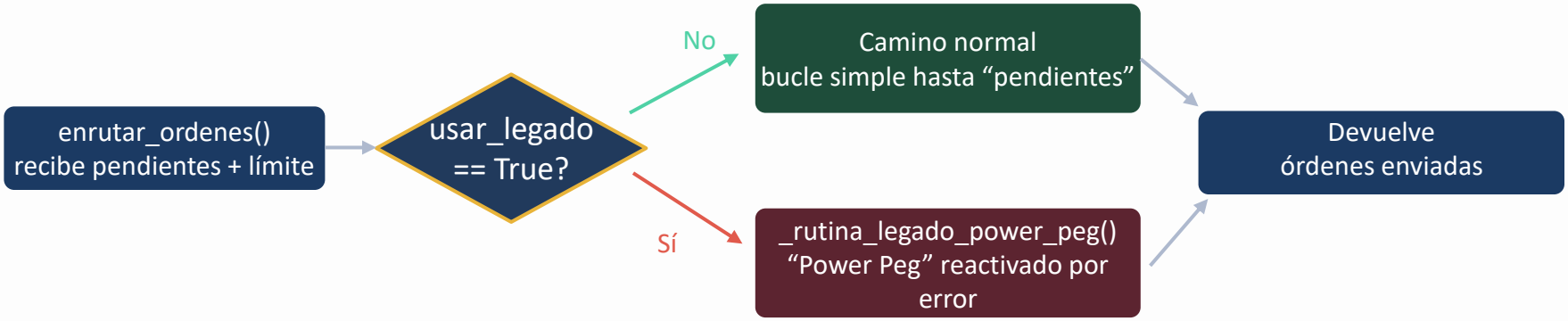


US$ 440 M	45 min	8 servidores	1
perdidos	duró el incidente	en el despliegue	servidor mal actualizado

Consigna: ¿qué técnica de esta unidad hubiera detectado el problema antes de llegar a producción?

Código simulado

El Código simula un enrutador de órdenes simplificado: recibe órdenes pendientes de un cliente y las envía, respetando (en teoría) un límite diario que fija la firma.



Consigna: Revisando este diagrama, ¿qué condición o combinación de condiciones nunca se pone a prueba en ninguno de los dos caminos? ¿Con qué técnica la encontrarían?